

Moukangwe L. (224139632)

Role: Sensor Management Module Developer

Introduction

The INT316D module, Internet Programming, has significantly improved my understanding of enterprise web application development. Before this semester, I only understood basic front-end development concepts such as HTML and CSS, but I did not fully understand how large-scale web systems operate behind the scenes. Through this module, I learned important enterprise technologies and concepts including the evolution of Java Enterprise Edition (JEE), Model-View-Controller (MVC) architecture, JavaServer Pages (JSP), Servlets, exception handling, Enterprise JavaBeans (EJB), Java Persistence API (JPA), JPQL, and application security. These concepts were applied practically in our semester project called SmartFlow, a traffic management system developed for the Witbank area. My role in the project was to develop the Sensor Management Module, which allows technicians and administrators to manage traffic sensors used throughout the city. This reflection explains my contribution to the project and how the different INT316D concepts were applied during development.

My Role in the SmartFlow Project

Traffic monitoring in Witbank relies heavily on sensors such as traffic cameras, radar units, and inductive loop sensors. These devices collect important data about vehicle speed, traffic flow, and congestion levels. However, there was no centralized system to manage these sensors effectively. Our SmartFlow system was designed to solve this issue by allowing traffic personnel to manage sensors digitally.

My responsibility was to develop the Sensor Management Module. I created a complete page that displays all available traffic sensors in a table format, including information such as the sensor name, location, type, installation date, and operational status. I also developed functionality that allows technicians to add new sensors to the system and remove outdated sensors when necessary.

One of the most important features I implemented was a stored procedure called `sp_AddSensorWithInitialLog`. When a new sensor is added, the stored procedure inserts the sensor details into the `traffic_sensors` table and automatically creates an initial record inside the `traffic_logs` table. This ensures that every sensor begins with a log history immediately after creation. The stored procedure affects two database tables from one operation, helping maintain data consistency.

I also included several non-textbox controls as part of the interface. These included a range slider for sensor sensitivity levels and a toggle switch for enabling or disabling auto-calibration mode. These controls improved the appearance and interactivity of the system while also meeting the project rubric requirements.

Application of INT316D Topics

Evolution of JEE

Learning about the evolution of JEE helped me understand how enterprise applications developed over time. Initially, web applications used CGI scripts, which were inefficient because every request created a new process. Servlets improved performance by running inside a container and handling requests through threads. JSP was later introduced to simplify

the presentation layer, while frameworks and enterprise technologies such as EJB and JPA improved scalability and maintainability.

In the SmartFlow project, I applied these concepts by separating the application into different layers. I used Servlets to process requests and JSP pages to display information to users. Understanding the evolution of JEE helped me appreciate why enterprise systems use structured architectures and reusable components.

Model-View-Controller (MVC)

The Sensor Management Module follows the MVC architecture. The Model consists of the Sensor Java class and the related database tables. The View is represented by sensors.jsp, which displays sensor information and user forms. The Controller is SensorServlet, which processes requests and communicates with the database.

When a technician adds a sensor, the servlet receives the request, validates the form data, and calls the stored procedure before forwarding the results back to the JSP page. Keeping the presentation layer separate from the business logic made the code easier to maintain and debug. If changes are required in the future, they can be implemented in one layer without affecting the others.

JSP and Servlets

JSP and Servlets formed the foundation of my module. The sensors.jsp page dynamically displays data retrieved from the database using JSTL and Expression Language. The page contains forms for adding sensors and tables for displaying sensor information.

The SensorServlet handles both GET and POST requests. The doGet method retrieves all sensors from the database and forwards them to the JSP page for display. The doPost method processes form submissions when a user adds or deletes a sensor.

Through this process, I learned how Servlets act as controllers that manage communication between the browser, the database, and the user interface. I also learned the importance of sessions in maintaining user login information throughout the application.

Exception Handling

Exception handling was an important part of my module because database operations can fail for many reasons. I implemented try-catch-finally blocks to manage possible errors during database access and stored procedure execution.

For example, if a technician attempts to add a sensor with a duplicate name, the system catches the SQL exception and displays a user-friendly message instead of crashing. If the database connection fails, the application redirects the user to an error page.

I also ensured that database resources such as connections, prepared statements, and result sets are closed properly inside the finally block. This prevents resource leaks and improves system stability.

Enterprise JavaBeans (EJB)

Although our project did not fully implement EJB technology, I studied its concepts and understood how it could improve the SmartFlow system. EJB simplifies enterprise development by providing built-in transaction management, security, and scalability.

For my module, a Stateless Session Bean could be used to manage sensor operations such as `addSensor()`, `deleteSensor()`, and `getAllSensors()`. Using EJB would reduce the amount of manual transaction handling required in the application. The container would automatically manage commits and rollbacks if an error occurs.

Learning about EJB helped me understand how large enterprise systems manage thousands of users and complex transactions efficiently.

Java Persistence API (JPA) and JPQL

The module also introduced me to JPA and JPQL. JPA provides object-relational mapping, which allows Java classes to represent database tables. Instead of manually writing SQL queries and mapping result sets, JPA automates much of the process.

In a future version of SmartFlow, the `Sensor` class could be annotated with `@Entity` and managed through an `EntityManager`. JPQL queries such as `SELECT s FROM Sensor s WHERE s.status = 'Active'` would make the application more database-independent and easier to maintain.

Although we used JDBC for simplicity, learning JPA showed me how enterprise applications reduce repetitive code and improve portability.

Application Security

Security was an important requirement for the SmartFlow project because only authorized users should access the sensor module. I implemented role-based access control using a `canAccessModule()` function that checks the user's role and permissions before granting access.

I also used prepared statements for all SQL queries to protect the application from SQL injection attacks. Session management was implemented to prevent unauthorized users from accessing pages without logging in.

In a real-world production environment, I would also implement HTTPS encryption and password hashing to improve security further.

Challenges and Solutions

One of the biggest challenges I faced was implementing the stored procedure that inserts data into two tables. Initially, I struggled to understand how `LAST_INSERT_ID()` works in MySQL. After researching and testing different approaches, I successfully implemented the procedure and connected it to Java using `CallableStatement`.

Another challenge involved deleting sensors that had related log records. Because of foreign key constraints, deleting a sensor could fail if logs existed. To solve this problem, I added validation checks before deletion.

I also found it challenging to implement the range slider and toggle switch because I had limited JavaScript experience. However, after practicing event handling and UI updates, I managed to create interactive controls successfully.

What I Learned Throughout the Semester

INT316D taught me much more than programming syntax. I learned how enterprise applications are structured, how databases interact with web systems, and how important architecture and security are in professional software development.

I also improved my teamwork and communication skills. Our group divided tasks according to each member's strengths, and we regularly assisted each other when problems occurred. Working on SmartFlow showed me the importance of collaboration, testing, and proper planning during software development.

The most valuable lesson I learned was the importance of database consistency and transactions. The stored procedure I implemented ensures that sensor data and log records remain synchronized, which is essential for reliable reporting.

Conclusion

The SmartFlow project gave me the opportunity to apply all the major concepts covered in INT316D to a real-world system. My Sensor Management Module demonstrates MVC architecture, Servlets, JSP, exception handling, database integration, stored procedures, and application security.

This project improved both my technical and problem-solving abilities, and it gave me practical experience in enterprise web application development. I am proud of the work I completed and confident that the knowledge and skills gained during this semester will help me in future software development projects and professional environments.